Software Requirements and Design Document

For

Group 2

Version 1.0

Authors:
Colin Maloney
Alex Nino
Kobus VanSteenburg
Samuel Pauly
Zachary Kupersmith

1. Overview (5 points)

Track Tracker is a music streaming data analytics tool. The tool will ingest songs from TikTok and corroborate data from Spotify to identify tracks with the potential to become future hits. We will be using publicly available APIs and web-scraping as needed to supplement the data. For backend services / data ingestion we are using Python and for front-end we are using Next.JS with TypeScript.

2. Functional Requirements (10 points)

1. **High – The system shall retrieve a list of trending TikTok videos at a configurable interval.**
2. **High – The system shall extract artist name and track title metadata from each TikTok video's associated sound object.**
3. **High – The system shall search the Spotify Web API using the extracted artist and track names.**
4. **High – The system shall persist all track, artist, and enrichment data in a PostgreSQL database.**
5. **High – The system shall prevent duplicate tracks and artists by enforcing uniqueness using the Spotify URI.**
6. **Medium – The system shall log all ingestion and enrichment failures.**

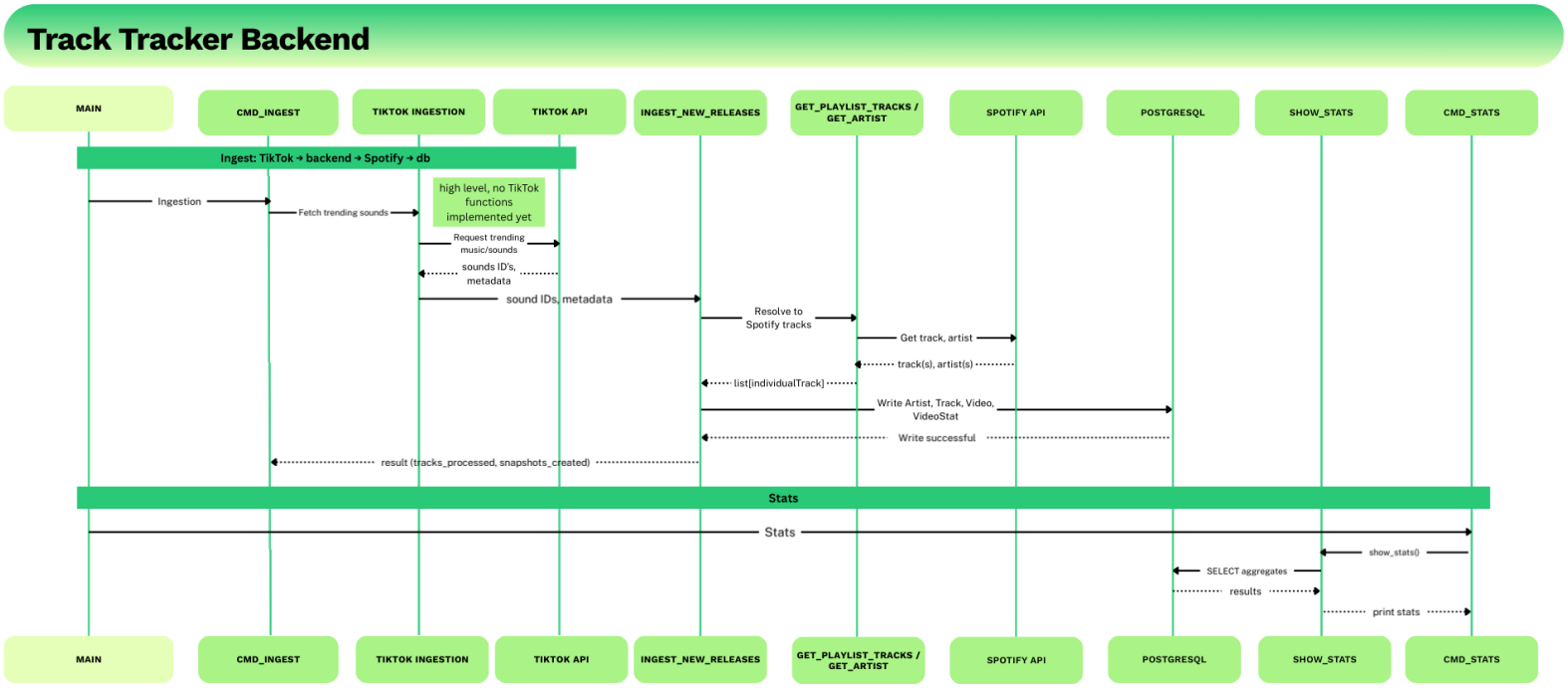
3. Non-functional Requirements (10 points)

1. **High – The system shall retry failed API calls up to three times using exponential backoff.**
2. **Medium – The system shall provide structured logging for all pipeline stages.**
3. **Medium – The system shall support horizontal scaling of enrichment workers.**

4. Use Case Diagram (10 points)

*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.*
***Textual descriptions of use cases**: For the first increment, the textual descriptions for the use cases are not required. However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.*

5. Class Diagram and/or Sequence Diagrams (15 points)



6. Operating Environment (5 points)

- **Backend:** Python 3.12+
- **Framework:** Fast API + Uvicorn / Next.JS
- **Database:** PostgreSQL 14+
- **Storage:** AWS RDS - PostgreSQL
- **External APIs:** TikTok-Api (unofficial), Spotify Web API
- **Version Control:** GitHub

7. Assumptions and Dependencies (5 points)

Assumption

- TikTok-Api remains functional and accessible.
- Spotify API rate limits are sufficient for projected volume.
- Extracted TikTok sound metadata contains usable artist/track names.
- Spotify URI uniquely identifies tracks.

Dependencies

- Spotify Web API availability.

- TikTok API stability.
- PostgreSQL database uptime.
- AWS infrastructure availability.
- Python package compatibility.